Stage 2 Plan: Building on the 1995 Thesis

> **Proposal, open for revision.** Stage 1 recovered the thesis, code and feature data. This page sets out how to build on them, in phases. Each phase has a goal, its inputs, what it produces, and a check that says whether it worked.

Starting point

What survives from 1995:

- **The thesis**, fully recovered and checked against the printed copy.
- **The code:** the 1995 MATLAB 4.0 programs and the author's 2015 refactor.
- **The RMC Speech Database feature files:** 2,547 formant tracks (F1–F3) from 14 speakers saying eight phrases, recorded on microphone and over telephone.

What does not:

- **The raw recordings.** The 1995 floppies hold only the period software (MS-DOS 6.22, Windows for Workgroups 3.11, MATLAB 4.0 and toolboxes, shareware audio tools). The `.WAV` archive did not survive. New audio is therefore needed for anything upstream of the formant tracks.

The formant tracks are exactly the input to the recognizer (`FARTMAPT.M`), which computes the formant ratios, normalizes them, warps them to a standard length and classifies them. The thesis's classification results can therefore be reproduced without the audio.

Where the world stands now (models, free databases, and the open problems) is surveyed in STATE_OF_THE_ART.md. Each phase below builds only where that survey finds a gap, and uses existing models and data everywhere else.

Phase 0: Reproduce the 1995 results

> **Done, 2026-11-08.** Every Fuzzy ARTMAP recognition rate reproduces exactly where the data survived complete, and four of those six tables match cell for cell. The TFRV backprop benchmark has now been run. Results and every discrepancy: PHASE0.md.

Goal. Port the recognizer and confirm that the recovered data reproduces Chapter 6. This is the baseline for everything after it, and it proves that Stage 1's recovery is complete.

Work.

- Port `FARTMAPT.M` (Fuzzy ARTMAP, with the TFRV and BFRV representations) and the backprop benchmark (`BPMAPPER.M`, `BPTESTER.M`) to **Python** and numpy (decided). The 2015 refactor, run under GNU Octave, serves as a cross-check.

- Rebuild the Chapter 6 protocol (§6.2):
  - **Speaker-dependent:** the first half of each speaker's samples train and the second half test, in recording order.
  - **Speaker-independent:** five tokens of each of phrases 2–9 per speaker, trained on ten speakers and tested on the eleventh, rotated eleven times.
  - Only the eleven speakers with at least 40 samples are used.
- Run the one experiment §6.2.2 left undone: the **backprop benchmark on the TFRV** representation.

Check. Every test reproduces the thesis results within a few percent:

Test	Microphone	Telephone
Fuzzy ARTMAP, TFRV, speaker-dependent (Tables 6.5-M, 6.5-T)	98.2%	91.5%
Fuzzy ARTMAP, TFRV, speaker-independent (6.6-M, 6.6-T)	94.1%	87.0%
Fuzzy ARTMAP, BFRV, speaker-dependent (6.3-M, 6.3-T)	97.1%	88.0%
Fuzzy ARTMAP, BFRV, speaker-independent (6.4-M, 6.4-T)	94.1% (printed; the table sums to 93.9%)	85.2%
Backprop, BFRV, speaker-dependent / independent (6.1, 6.2)	91.7% / 88.7% (printed; 391/440 is 88.9%)	—

Any test that does not reproduce gets an explanation, not a tuned parameter.

Phase 1: New audio

Goal. Audio for the eight thesis phrases from many more voices, accents and languages, in both microphone and telephone form, with each source's licence and consent on record.

Sources, real speech first.

- **Real speech.**
  - **TIMIT.** Phrase 2, "*She had your dark suit in greasy wash water all year,*" is TIMIT sentence SA1, read by all 630 TIMIT speakers from eight US dialect regions. Phrase 3 is close to TIMIT's "*Don't ask me to carry an oily rag like that.*" TIMIT is licensed through the LDC, which charges for it.
  - **Speech Accent Archive.** A single paragraph ("Please call Stella…") read by well over a thousand speakers of a few hundred native languages, covering accents of the same text. Free for non-commercial use with citation; confirm the current licence.
  - **Mozilla Common Voice.** Real read speech in well over 100 languages, released CC0. It does not contain the thesis phrases, so it serves language and accent work rather than phrase recognition.
- **New recordings.** The 1995 method still works: volunteers read the eight phrases, now with written consent and an open licence. A simple web recorder could collect them, giving an *RMC Speech Database 2* that can be published like the original.
- **Synthetic voices (augmentation only).**
  - TTS can generate the eight phrases in many voices, accents and languages cheaply. That is useful for stress-testing, but a recognizer trained only on synthetic speech generally does not transfer well to real speech.

    - **ElevenLabs cannot be used for this without permission.** Its Prohibited Use Policy forbids using its output to develop, train, fine-tune or improve any AI model, or including it in a dataset for those purposes, unless the applicable terms expressly allow it. Using it would need written agreement from ElevenLabs.
    - The alternative is an open-weight TTS model whose licence permits training use, checked model by model.
    - In either case the 1995 volunteers' voices are not cloned. They consented to being recorded, not to synthesis.

Telephone channel. Any wideband recording can be given a telephone version by simulation: a 300–3300 Hz band-limit, resampling to 8 kHz, and μ-law (G.711) coding, optionally with line noise. Every new sample can then exist on both channels, as in 1995.

Check.

- A data card for each source: licence, consent, speaker count and accent coverage.
- A synthetic-to-real transfer test: train on synthetic speech, test on real speech, and report the drop in accuracy.

Phase 2: Formant ratios at full resolution, and against modern features

Goal. Answer the question the thesis left open: does the speaker-independence of formant ratios hold up with many more speakers, accents and languages?

Work.

- **Re-run the full 1995 pipeline on new audio** (spectrogram, noise mask, silence reduction, formant tracking, TFRV), keeping the detail the 1995 processing budget discarded. That means finer time and frequency resolution, more formants, and no fixed-length cut.
- **Add the §7.3 future-work items:**
    - frequency normalization that adapts to each telephone connection;
    - a noise mask tailored to each sample;
    - a higher sampling rate on the telephone channel.
- **Compare on the same splits:**
    - TFRV + Fuzzy ARTMAP;
    - MFCCs with a standard classifier;
    - a self-supervised speech embedding (a wav2vec 2.0, HuBERT or WavLM-type model).

Check. A results table across speakers, accents and channels. The thesis claim stands if TFRV's accuracy drop from speaker-dependent to speaker-independent testing stays small as the speaker pool grows.

Phase 3: Language identification first, then a language-specific recognizer

Goal. The architecture the author intended in 1995: identify the language first, then load that language's recognizer.

The written thesis plants the seed rather than stating the design:

- §4.4.1 calls for a system that learns "independent of speaker type or spoken language";
- §7.2 credits Fuzzy ARTMAP's localized memory with supporting "modular packaging and reconfiguration of active vocabulary."

Spoken language identification itself is now mature, with public models and datasets such as VoxLingua107 (107 languages, CC BY 4.0). It is the **baseline here, not the contribution**.

Work.

- An existing language-identification model as the front end.
- One Fuzzy ARTMAP module per language, loaded on demand, using ARTMAP's incremental learning to add a language or vocabulary without retraining the others.

The modern counterpart of 1995's "modular packaging" is a **small adapter per language** (LoRA) on a shared pretrained speech encoder, swapped in when the language is identified. Running both versions is a direct test of the 1995 design against today's.

Check. End-to-end accuracy against a single multilingual recognizer. Then add a new language and measure how much the accuracy on the existing languages changes; with ARTMAP it should not fall.

Phase 4: Toward a universal vocalization model (exploratory)

The vision. Recognize communicative sound across humans and other animals, and infer meaning from the situations in which a pattern is used.

Where the field is. This is an active research frontier, not open ground:

- **Earth Species Project's NatureLM-audio (2024)** classifies, detects and captions animal sounds across species. Its makers say plainly that it is not a translator.
- **Project CETI (2024)** found combinatorial structure in sperm-whale codas, a "phonetic alphabet," but not yet their meaning.
- **Prat et al. (2017)** published about 90,000 Egyptian fruit bat calls annotated with who was involved, what they were doing and the context. This is exactly the kind of data that ties sound to situation.

What this project could add. Two threads come straight from the thesis:

1. **Size-invariant features across species (a hypothesis).**
    - Formant ratios cancel out vocal-tract length between human speakers.

    - Across animals, vocal-tract length scales with body size, and formant spacing tracks it (Fitch 1997, *JASA* 102:1213). The same normalization might therefore carry across species and body sizes.
    - This is untested and worth testing.
2. **Open-ended learning.**
    - Adaptive Resonance Theory was built to solve the stability–plasticity dilemma: it learns new categories without forgetting old ones.
    - That fits discovering unknown call types in data that never stops arriving.

First experiment. On the fruit-bat dataset, predict the annotated context from the call's acoustics, with and without ratio-type features, using Fuzzy ARTMAP and a modern classifier. Keep the claim to what the result shows.

Training techniques: the 1995 claims against today's methods

Each phase pairs the 1995 method with a current one, so the comparison is the result:

1995 claim	Today's counterpart	Test
Formant ratios are speaker-independent	Self-supervised speech encoders (wav2vec 2.0, HuBERT, WavLM)	Phase 2: accuracy across speakers, accents and channels
Fuzzy ARTMAP learns new data without forgetting the old	Fine-tuning, which is prone to catastrophic forgetting; replay and adapters as mitigations	Add phrases, speakers and languages one at a time; measure accuracy on earlier ones
Fast training, no random initial weights	Parameter-efficient fine-tuning (LoRA) of a pretrained model	Training time and accuracy, reported side by side
Modular vocabulary packaging	One adapter per language on a shared encoder	Phase 3
Speech recorded over a real telephone line	Augmentation: telephone codec simulation, noise and room acoustics, SpecAugment	Phase 1 synthetic-to-real and channel tests

The 1995 testing controls (§6.2.1) already match modern evaluation practice: no overlap between training and test phrases, chronological splits, nothing discarded to improve results. They stay in force, plus whole-speaker hold-outs and fixed random seeds.

Practices shared with bug-bash

bug-bash, the author's other public project, measures and trains how well language models find bugs. These of its practices carry over:

- **A research journal.** `JOURNAL.md`, one entry per session, using the same template, including the *Caught in review* line. That line records AI-written code or tests that were rejected or corrected, and why.
- **A plan with a *done when* column.** Each phase above becomes a row with a concrete exit test.

- **Automatic, judge-free scoring.** Every result is a number computed from held-out data. Any attempt to game a metric (for example, leakage between training and test speakers) is logged, as bug-bash logs reward hacking.
- **CI with a score gate.**
  - GitHub Actions runs the tests on a Python and OS matrix.
  - Phase 0's reproduction of Chapter 6 becomes a **required check**: a pull request that moves any reproduced result outside its tolerance fails, and the change in score is posted on the pull request.
  - A nightly run re-scores everything.
- **Reproducibility.** A pinned environment and a Docker image for the experiments.
- **Data and model cards.** Licence, consent and coverage for every dataset; training data and known limits for every model.
- **A metrics history.** Every scored run is appended, and the trends are published on a small GitHub Pages dashboard.
- **An agent harness over MCP.** BLINDMAN's listening abilities are exposed as tools (for example `listen`, `identify_language`, `recognize`, `press`), so an agent such as Claude Code can be tested navigating by sound alone.
- **A division of work and an *Author and tools* statement.** These say which decisions are the author's and what was built with AI assistance.

A shared environment: `deafman` and `blindman` modes for `ivrsim`

bug-bash's IVR simulator (`ivrsim`) is entirely fictional, and its design lists speech in and out as an optional later release. It would gain two **output modes**. Each delivers the same call flow through one sense only:

Mode	The caller receives	Serves	Agent test
`deafman`	Text only: prompts as text, never audio	Callers who are deaf or hard of hearing, as text relay and real-time text (RTT) serve them on real phone lines	A language-model caller reading text, which is bug-bash's release 1 as designed
`blindman`	Speech only: prompts as audio, never text	Callers who are blind or have low vision; a phone menu is already audio-first	BLINDMAN: it must recognize the prompts by ear before it can choose what to press

What changes and what doesn't:

- **The engine is unchanged.** The modes are a presentation layer over `IvrSession.next()`. `deafman` passes the prompt text through. `blindman` renders the prompt to audio with text-to-speech, and later optionally with recorded prompts, and withholds the text.
- **Input stays the same in both modes:** keypad digits, plus speech in `blindman` mode once speech input exists.
- **The trace still records everything,** so grading is identical in both modes.

Why it links the projects:

- The same fictional call flows can be run in both modes.
- The gap between a text caller and an audio caller then measures exactly what hearing adds or costs. For example, a prompt that reads clearly but is misheard when spoken (a homophone, or a number read too

fast) is a defect that only `blindman` mode exposes.
- Both modes test accessibility, the shared motivation of the two projects.

How the code is divided:

- `ivrsim` lives in bug-bash and owns the engine and the two modes.
- BLINDMAN supplies the *ear*: the recognizer that turns `blindman`-mode audio back into something an agent can act on.
- BLINDMAN uses `ivrsim` as a dependency and never copies it. That keeps to the decision that IVR client work stays out of this repository.

Timing: after Phase 3, when BLINDMAN can recognize speech in general. The text-to-speech used for `blindman` mode should be an open model, for the licensing reasons in Phase 1.

How this serves BLINDMAN

The AUI uses dimensions of sound to carry meaning to a listener. BLINDMAN must do the reverse: hear those dimensions and recover the meaning. Each phase builds part of that ear:

- recognizing phrases on any voice and channel (Phases 0–2);
- knowing which language it is hearing (Phase 3);
- eventually, interpreting sound it was never taught (Phase 4).

Open decisions

- ~~Language for the port~~: **Python** (decided 2026-11-08).
- ~~Timing~~: **Stage 2 runs in parallel with bug-bash** (decided 2026-11-08).
- ~~The shared `ivrsim` environment~~: **adopted**, with `deafman` and `blindman` output modes (decided 2026-11-08). The modes still have to be added to bug-bash's `ivrsim` design.
- **Synthetic speech:** seek written permission from ElevenLabs, or use an open-weight TTS model whose licence allows training.
- **Data budget:** whether to license TIMIT through the LDC, and whether to run a new volunteer-recording drive.